

Week 3, Day 1 — student handout

Umesh Indrajith · Keep this sheet. Two labs today. SMS still starts week 4.

TODAY'S OUTCOMES

- if, short statement, switch (including tagless).
- for / range / break / continue.
- Slices: len, cap, make, append, backing-array gotcha.
- Maps: make, comma-ok, range (order not promised).
- Functions enough to ship helpers; return x, err as a copyable pattern.
- Green tests for Contains, Unique, Freq.

FOLDER FOR TODAY

```
mkdir -p ~/epic-go/week03/slicekit  
cd ~/epic-go/week03
```

VS Code: File → Open Folder → ~/epic-go/week03. gofmt on save is still required.

CONTROL FLOW

```
if n := len(name); n == 0 {  
    fmt.Println("empty")  
}  
  
switch day {  
case "Sat", "Sun":  
    fmt.Println("weekend")  
default:  
    fmt.Println("weekday")  
}  
  
for i, v := range courses {  
    fmt.Println(i, v)  
}
```

SLICES

```
s := make([]string, 0, 8)  
s = append(s, "SE")           // always assign the result  
  
a := []int{10, 20, 30, 40}  
b := a[:2]  
b = append(b, 99)           // may change a[2] — shared backing array
```

Slice header = pointer + length + capacity. Need independence? Copy into a new backing array.

MAPS

```
count := make(map[string]int)
count["SE"]++
v, ok := count["SE"]           // comma-ok: missing vs present
if !ok { /* not found */ }
delete(count, "SE")
```

Writing to a nil map panics. Range order is randomised — do not assert print order in tests.

LAB PACKAGE

```
cd ~/epic-go/week03/slicekit
go mod init epiclearn/week03
# implement Contains, Unique, Freq + tests
go test
```

WORDS TO REMEMBER

Term	Meaning today
Backing array	Storage underneath one or more slices
cap	How far append can grow without reallocating
Comma-ok	<code>v, ok := m[k]</code> — was the key present?
Tagless switch	<code>switch { case cond: }</code>
–	Blank identifier — intentionally unused

IF IT BREAKS

- if score → need a bool comparison
- append ignored → assign: `s = append(s, x)`
- nil map write → make first
- flaky map test → assert keys, not print order
- go test no module → go mod init

HOMEWORK (BEFORE NEXT CLASS)

1. Eight short answers in `exercises.md`.
2. Harden `Contains` / `Unique` / `Freq` with the extra cases on the homework sheet.
3. Optional: five lines on Effective Go slices/maps.